Section 1: Edge Impulse Results

Public Edge Impulse project link: https://studio.edgeimpulse.com/public/994176/live

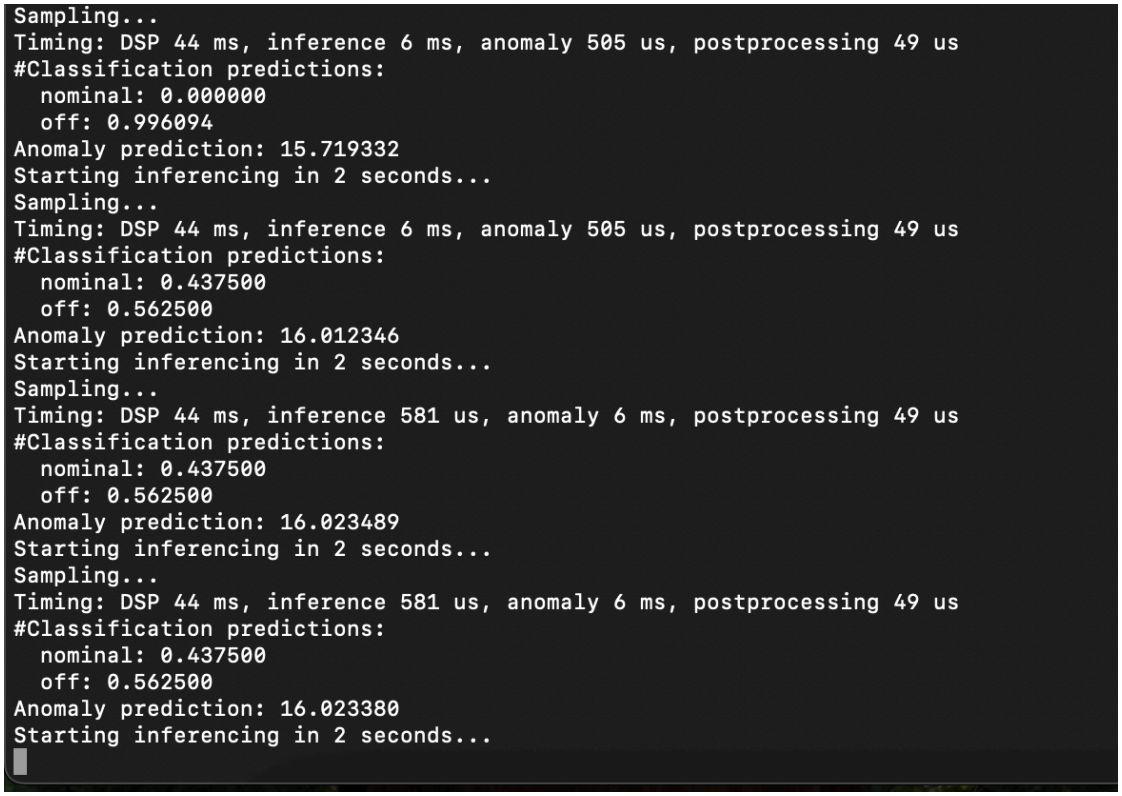
1. Raw inputs: State the raw inputs used in the Edge Impulse project.
The raw inputs are the time-series sensor data from three accelerometer axes: accX, accY, and accZ, with a 2,000 ms window and 220 ms stride.

2. NN classifier features: State the type of features passed into the NN classifier, the number of input features, and at least five actual feature names used to train the classifier.
The types of features passed are time-domain features and frequency-domain features. There are 33 input features. Some of the features include: accX Peak 1 Freq, accY Spectral Power 1.0 - 2.0 Hz, accZ Peak 2 Height, accX RMS, and accZ Spectral Power 2.0 - 5.0 Hz.

3. NN classifier outputs: State the outputs of the NN classifier.
The NN classifier has 2 outputs, which are the classes nominal_motion and off_sample.

4. Anomaly detector features: State the type of features passed into the anomaly detector, the number of features used, and the feature names used to train the anomaly detector.
The types of features passed are time-domain features and frequency-domain features. There are 4 input features. The features are accX RMS, accX Peak 1 Height, accY Spectral Power 2.0 - 5.0 Hz, and accZ Spectral Power 2.0 - 5.0 Hz

5. Deployment evidence: Include a screenshot showing the Edge Impulse model running on the Arduino Nano 33 BLE Sense and producing inference results.



6. Short interpretation: Briefly interpret the observed deployment results. Discuss whether the classifier output should always be trusted and under what conditions it may or may not be reliable.

The observed results show the model transitioning from a high-certainty state to an uncertain, anomalous state.

In the first reading, the classifier is highly confident (99.6%) that the device is in the "off" state.

In the next readings, the classifier becomes uncertain, splitting the probability almost evenly between nominal (43.7%) and off (56.2%).

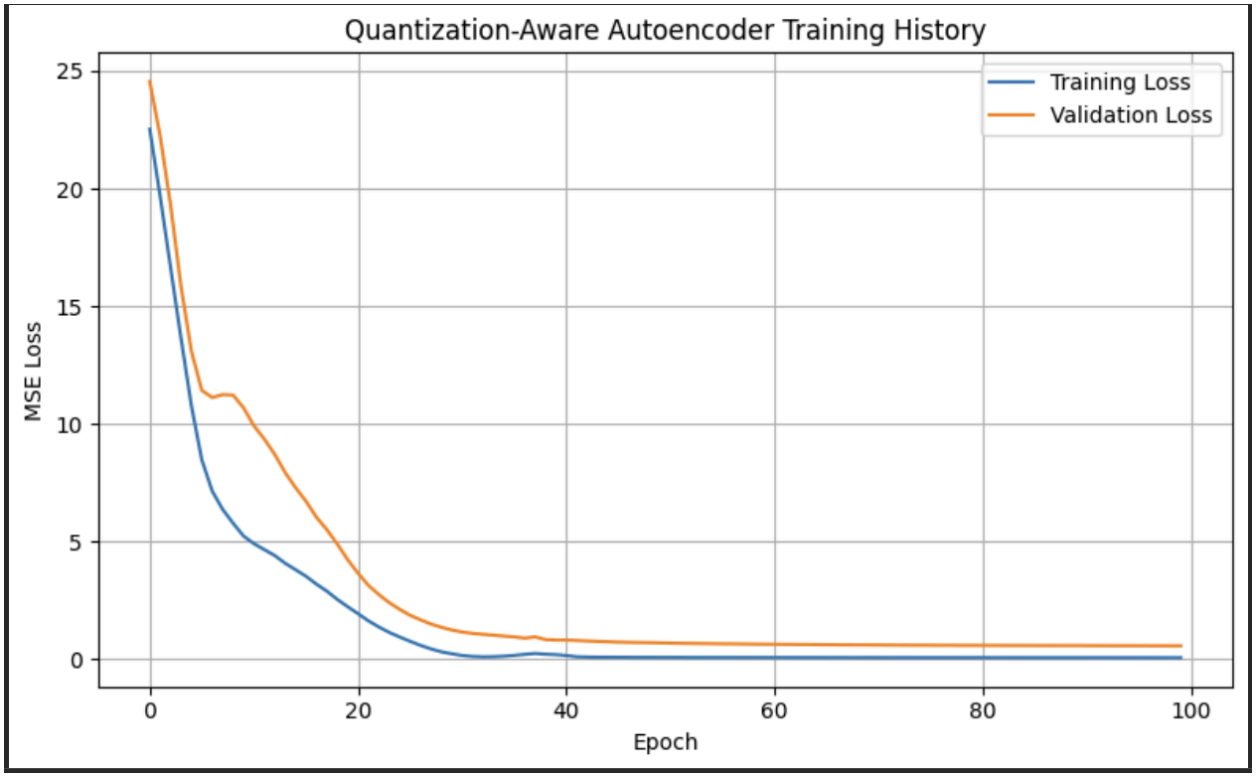
The Anomaly prediction is very high (~16.02). Since k-means anomaly scores typically indicate distance from normal clusters, a score this high suggests the vibration pattern being sensed does not match any of the data used during training.

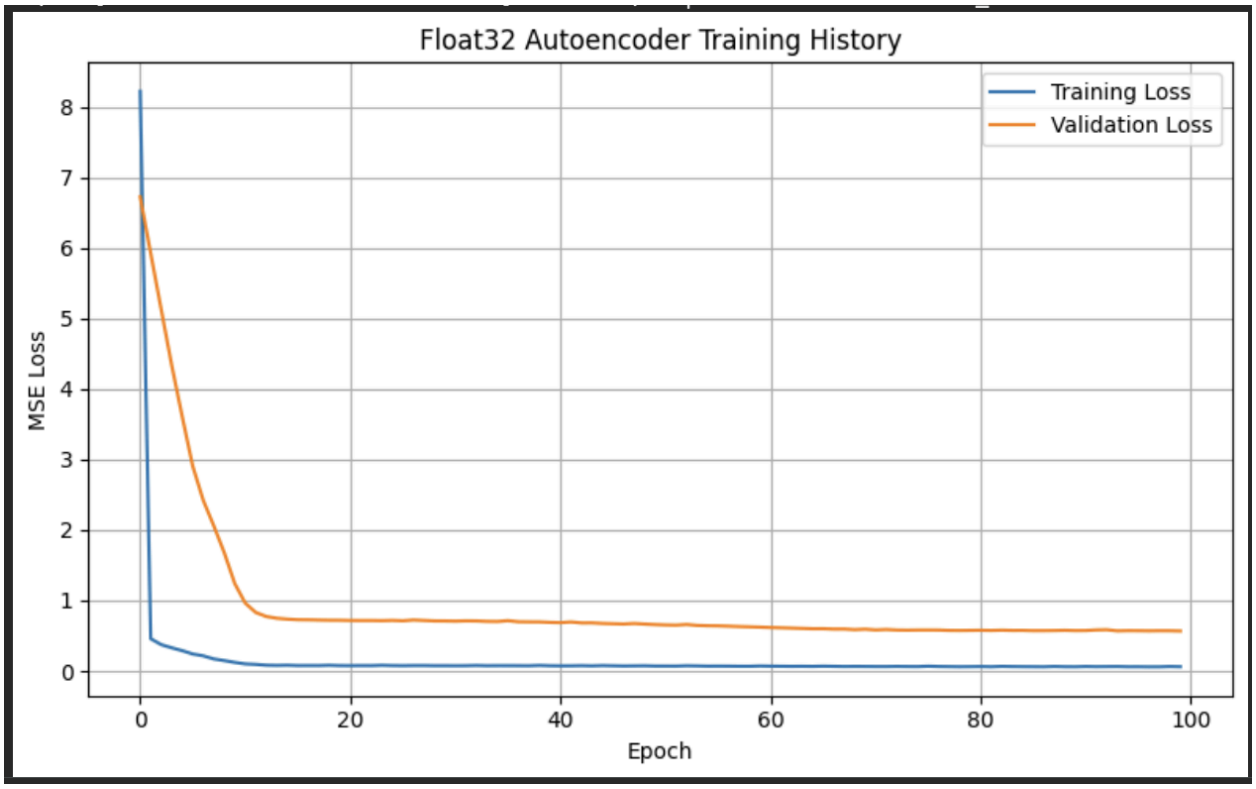
No, the classifier output should not always be trusted, especially when the anomaly score is high.

In order for the classifier output to be reliable, there should be a Low Anomaly Score, High Confidence/Softmax Margin, and Environmental Consistency

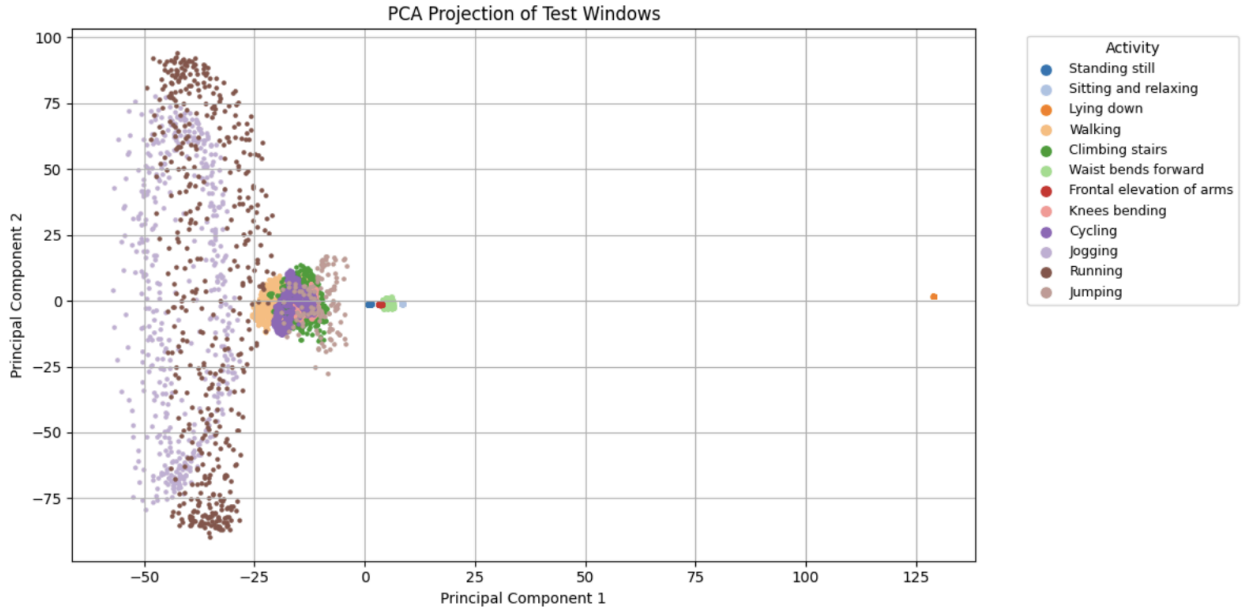
Section 2: Autoencoder Model Results

Training and validation loss curves:

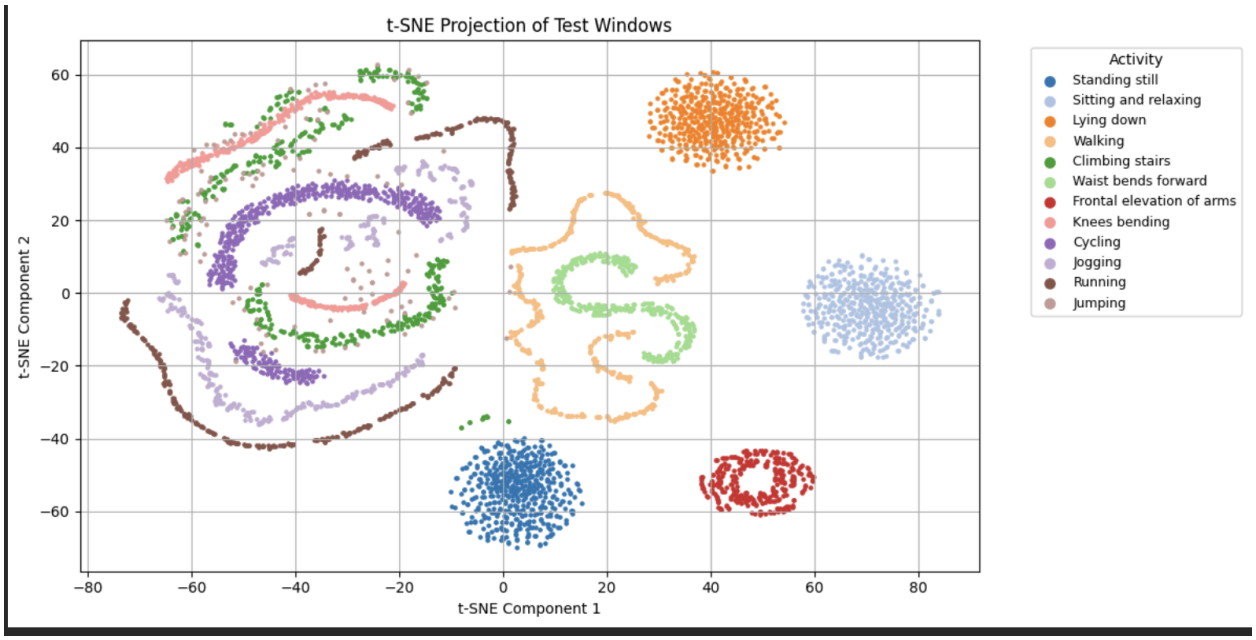




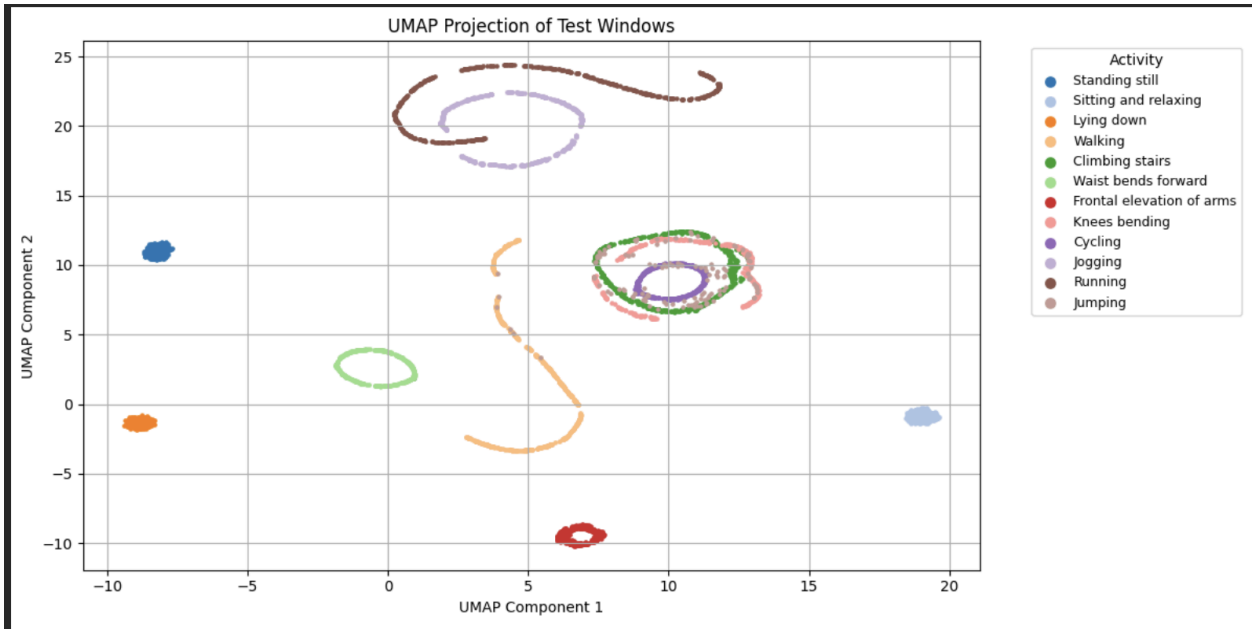
PCA visualization:



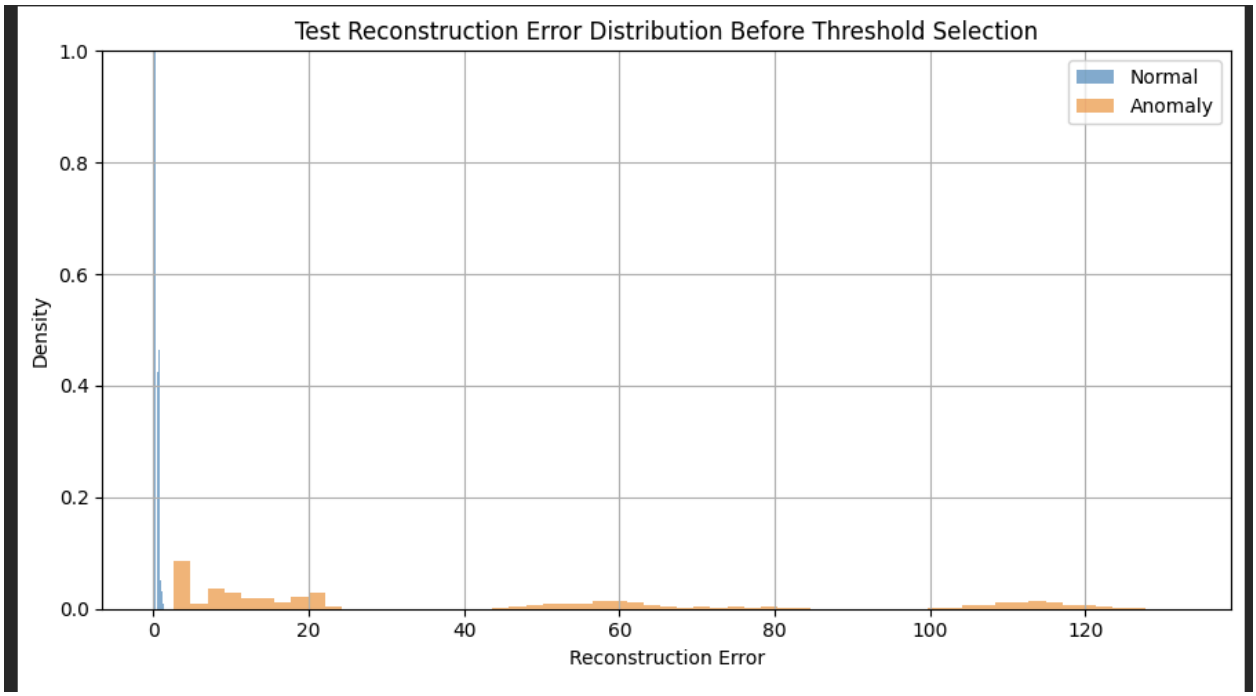
t-SNE visualization:



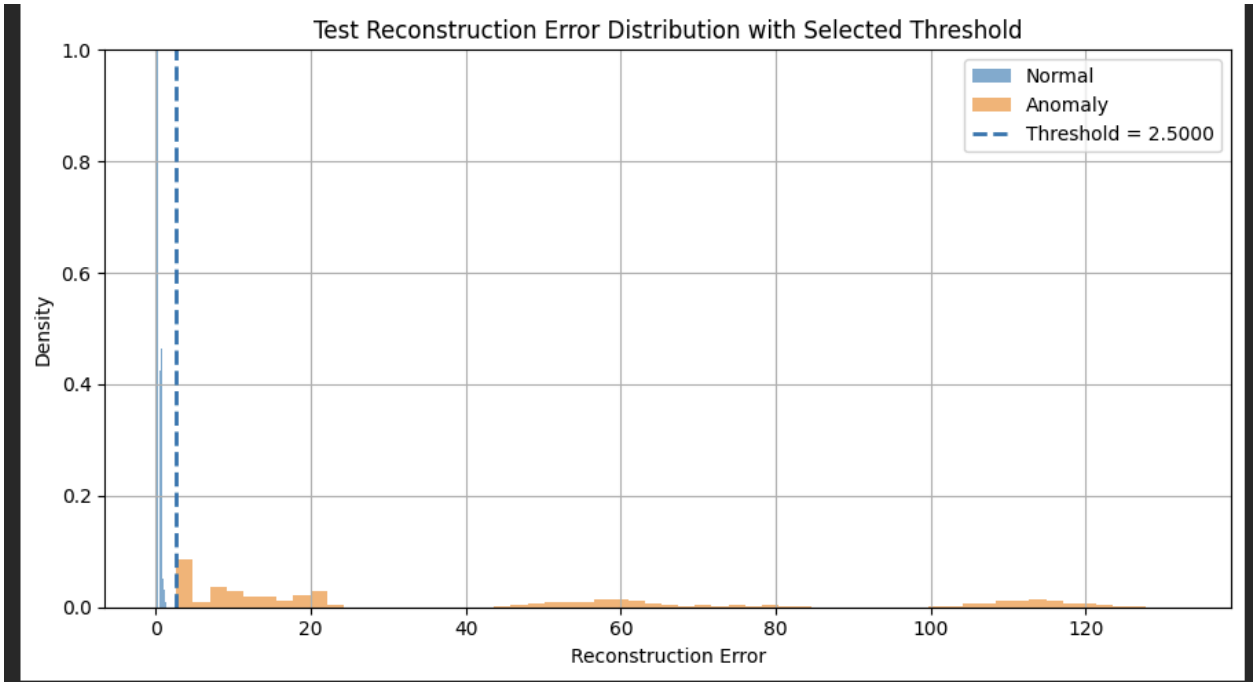
UMAP visualization:



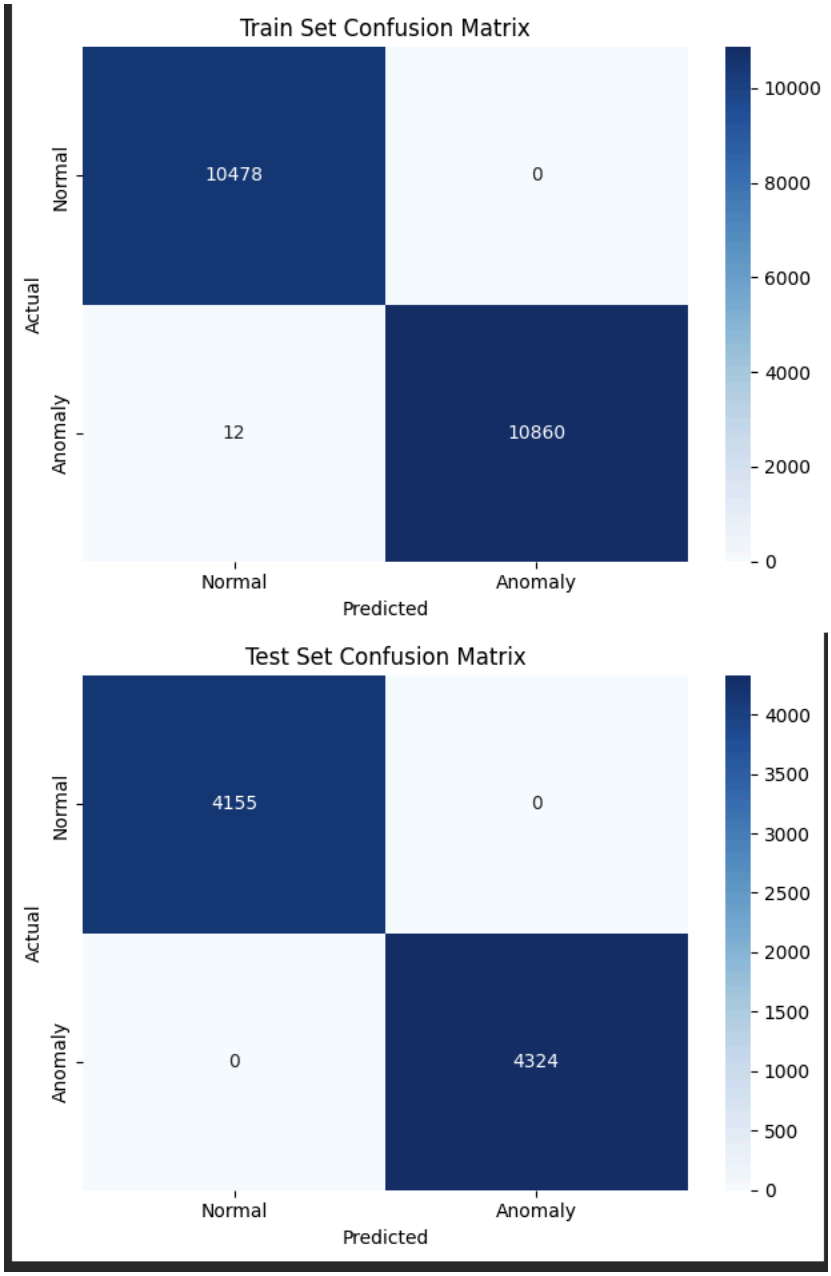
Reconstruction error distribution

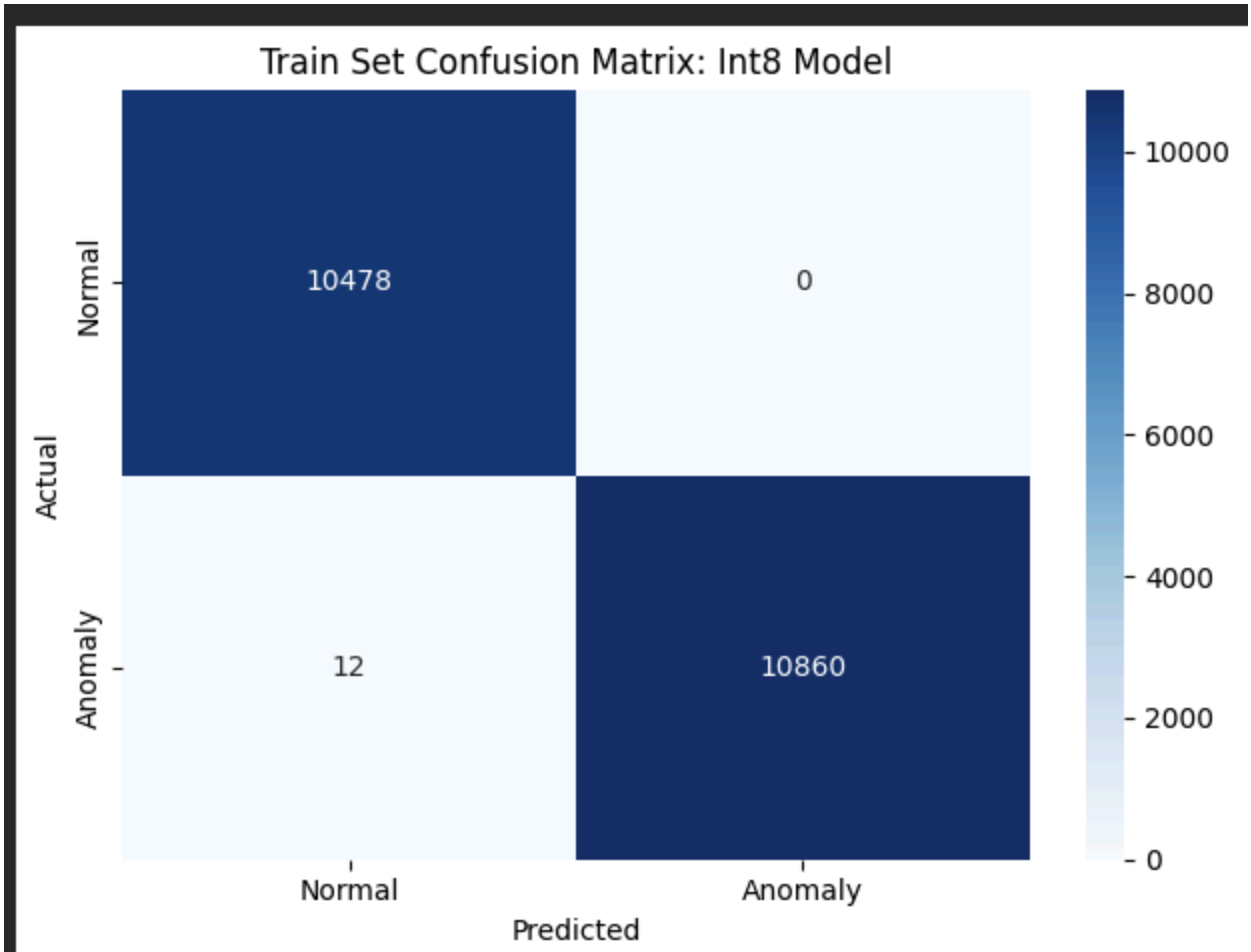


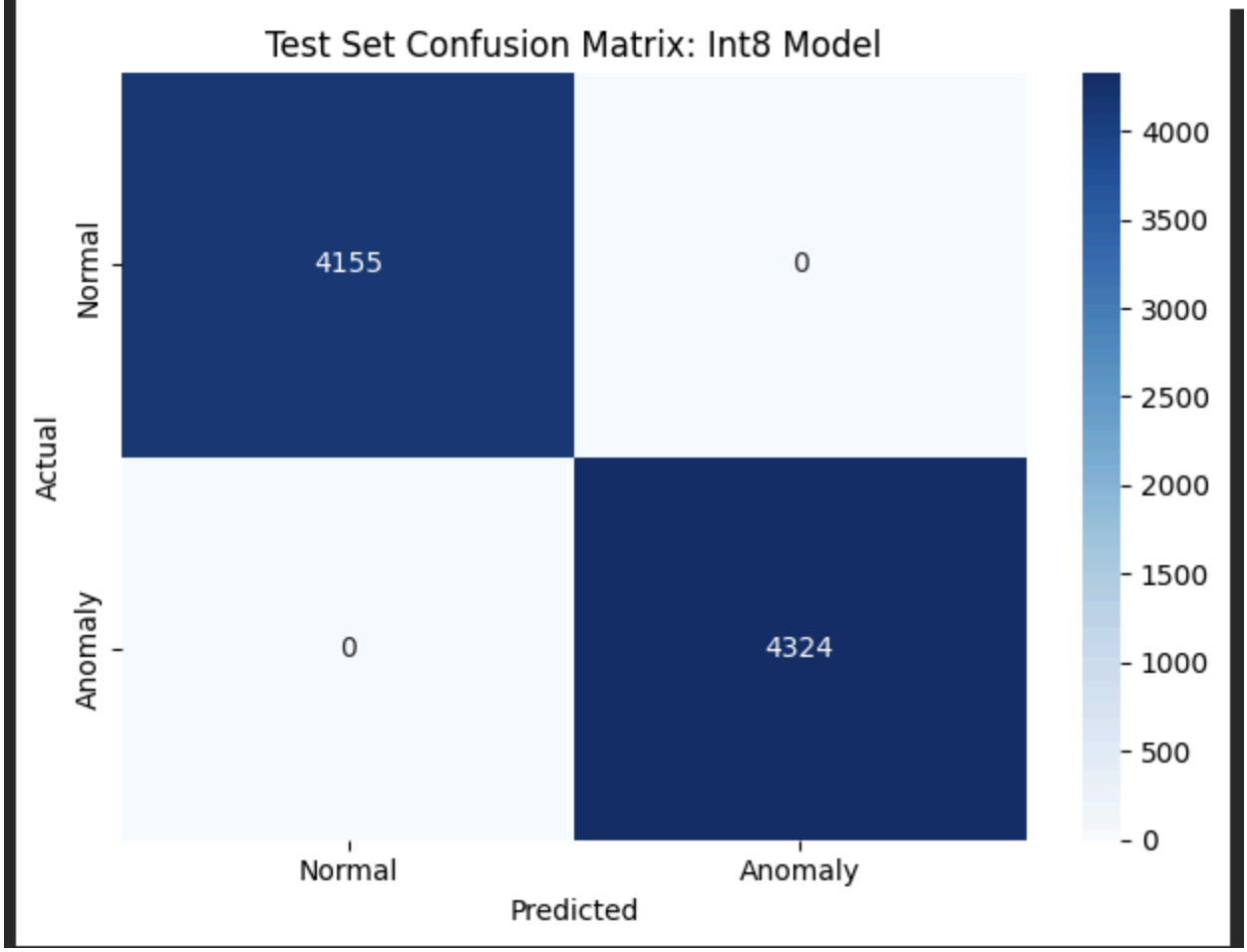
Selected anomaly threshold:



Confusion matrix:


Train Set Confusion Matrix
Actual
Normal
Anomaly
10478
0
12
10860
Normal
Anomaly
Predicted
10000
8000
6000
4000
2000
0
Test Set Confusion Matrix
Actual
Normal
Anomaly
4155
0
0
4324
Normal
Anomaly
Predicted
4000
3500
3000
2500
2000
1500
1000
500
0

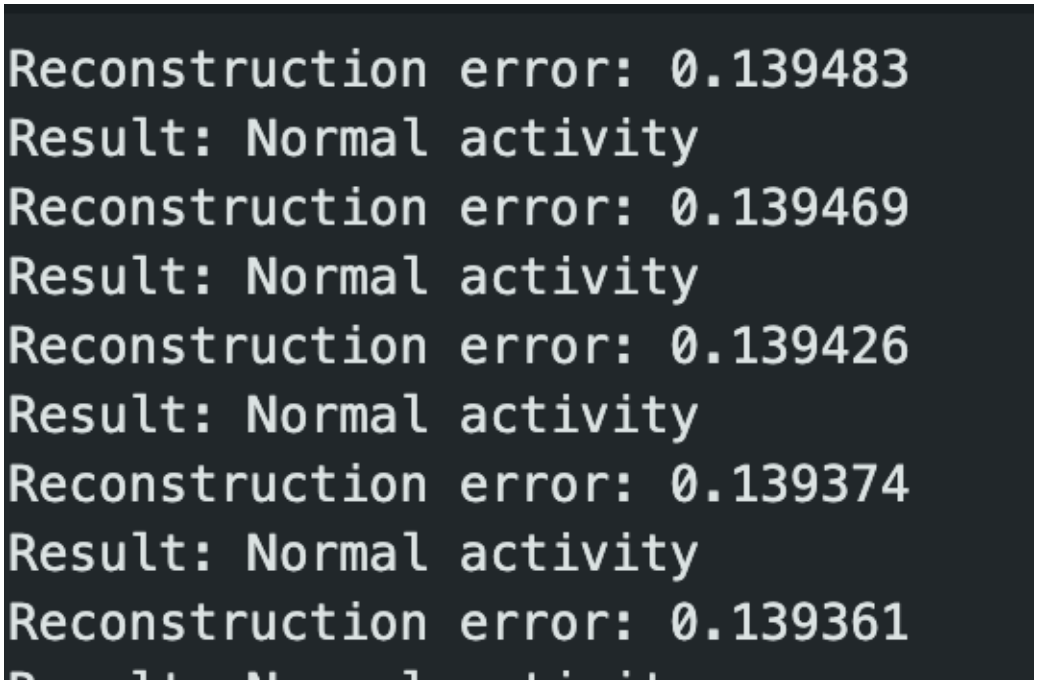

Train Set Confusion Matrix: Int8 Model
Actual
Normal
Anomaly
10478
0
12
10860
Normal
Anomaly
Predicted
10000
8000
6000
4000
2000
0


Test Set Confusion Matrix: Int8 Model
Actual
Normal
Anomaly
4155
0
0
4324
Normal
Anomaly
Predicted
4000
3500
3000
2500
2000
1500
1000
500
0

Classification report or evaluation metrics:

```
Train set evaluation: int8 model
              precision    recall  f1-score   support

  Normal (0)       1.00      1.00      1.00     10478
 Anomaly (1)       1.00      1.00      1.00     10872

    accuracy                           1.00     21350
   macro avg       1.00      1.00      1.00     21350
weighted avg       1.00      1.00      1.00     21350

Test set evaluation: int8 model
              precision    recall  f1-score   support

  Normal (0)       1.00      1.00      1.00      4155
 Anomaly (1)       1.00      1.00      1.00      4324

    accuracy                           1.00      8479
   macro avg       1.00      1.00      1.00      8479
weighted avg       1.00      1.00      1.00      8479
```

Arduino serial monitor output showing reconstruction error and normal/anomaly detection results



Section 3: Discussion Questions

1. What threshold did you choose for anomaly detection, and why?

I chose a threshold of 2.5. Based on the Reconstruction Error Distribution plot, the "Normal" activities (blue) all have errors below 2.0, while the "Anomalies" (orange) start at 4.0 and higher. Choosing a threshold of 2.5 places the boundary in between these two clusters. This provides a safety margin that prevents minor sensor noise from being flagged as an anomaly (avoiding False Positives) while still being sensitive enough to catch every jumping or running motion.

2. How does reconstruction error help distinguish normal and anomalous activity?

Reconstruction error measures how closely the autoencoder can recreate the input data.

As the model is trained only with the data set of the regular activities, therefore, it becomes aware of the characteristics of those movements. So, it will be able to reconstruct these movements with high accuracy and will have low error. For anomaly detection, as the system has not encountered the previously unseen movements, it does not know how to represent it, and hence there will be poor representation of these

actions, thereby leading to high error. It means that an anomaly detection process can be achieved mathematically using error thresholding.

3. What information from the Python notebook must be preserved when deploying the model to Arduino?

To ensure the model works exactly the same on the hardware as it did in the notebook, three things must be preserved: model weights, threshold value, and scaling parameters.

4. Why is it important for the Arduino sketch to use the same preprocessing assumptions as the notebook?

These machine learning algorithms are sensitive to the format of the data. If the model was trained on a sliding window size of 100 observations and the flattened 300-dimensional vector by your Jupyter Notebook, but the Arduino feeds 50 observations or no flattened vectors at all, the calculations within the neural network won't work or will give you gibberish.

The same way, if the machine learning algorithm is expecting data in "Gs" (gravitational units), but the Arduino feeds it data in raw "bits," the reconstruction error will be astronomical; all data will be classified as anomalies.

5. What are the main limitations of this anomaly detection method on a microcontroller?

Memory constraints because they might exceed the SRAM of a microcontroller like the Nano 33 BLE. Processing overhead is another potential limitation because calculating the reconstruction error requires running the full model which takes more power and time than a simple classifier. Rigidity of the method may also be a limitation, ff the user's normal behavior changes slightly, the model might start triggering false anomalies because it has no way to re-learn or update itself on the edge.